
Percona Operator for PostgreSQL on Kubernetes

Captured Procedures & Error Signatures —
Field Playbooks for Day-2 PostgreSQL Operations

Compiled for	Cash — Lead DBA / PostgreSQL SME
Date	03 August 2026
Contents	GitOps/ArgoCD, migrations, upgrades, incidents
Sourcing	Percona blog + Community Forum, restructured
Document	2 of 2 — companion to the Reference Index

Conventions & how these notes were built

Each section below is a self-contained operational note: the goal, the exact commands and custom-resource fields, and — most importantly for troubleshooting — the **literal error signatures** you will see when something is wrong, with the fix. Procedures were reconstructed from the Percona engineering blog and Percona Community Forum threads, then rewritten as operator-neutral runbook steps. Verify version-specific field names against the docs for the operator version you run.

Cluster shape assumed

Notes assume a v2/v3 Percona operator managing a Patroni-based cluster with a pgBackRest repo and PgBouncer, matching a typical 2–3 pod streaming-replication deployment. Adjust selectors and paths to your namespace and cluster name.

Version compatibility trap (read first)

The v2/v3 operator custom resources are **not** backward compatible with v1. When installing or upgrading via Helm, always pin the chart version explicitly or you risk an accidental major jump:

```
helm install my-operator percona/pg-operator --version 3.0.0 \
--namespace my-namespace --create-namespace
```

1. GitOps deployment with ArgoCD

Target flow: CI pushes manifests to Git → ArgoCD diffs against the cluster → ArgoCD creates/updates the operator and the PerconaPGCluster CR → the operator runs day-1 and day-2 tasks from those CR changes. Two operator-specific adjustments are required for ArgoCD to work cleanly.

1.1 Install and expose ArgoCD

```
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml

# expose the API server (LoadBalancer shown; ingress is fine too)
kubectl patch svc argocd-server -n argocd -p '{"spec": {"type": "LoadBalancer"}}'
kubectl -n argocd get svc argocd-server

# CLI login
argocd admin initial-password -n argocd
argocd login <EXTERNAL-IP>
```

1.2 Fix #1 — Server-Side Apply for oversized CRDs

Percona relies on OpenAPI v3.0 validation, which inflates the CRD manifests. A plain client-side apply hits the Kubernetes annotation size ceiling and fails with:

Error signature

```
Error from server (Invalid): error when creating "deploy/bundle.yaml":
CustomResourceDefinition.apiextensions.k8s.io "perconapgclusters.pg.percona.com" is invalid:
metadata.annotations: Too long: must have at most 262144 bytes
```

Fix: use Server-Side Apply. In GitOps, set it declaratively on each CRD via an ArgoCD sync option annotation:

```
kind: CustomResourceDefinition
metadata:
  annotations:
    argocd.argoproj.io/sync-options: ServerSideApply=true
name: perconapgclusters.pg.v2.percona.com
```

Manual applies too

Outside ArgoCD, the same fix is `kubectl apply --server-side -f bundle.yaml`. The migration playbooks in Parts 2–3 use `--server-side` for exactly this reason.

1.3 Fix #2 — Sync waves for ordering

The operator (CRDs + Deployment) must exist before the cluster CR is applied, and on teardown the CR must be removed first. Encode this with ArgoCD sync-waves — lower numbers apply earlier, and the order reverses on delete.

```
# operator bundle resources -> wave 1
metadata:
annotations:
argocd.argoproj.io/sync-wave: "1"
---
# PerconaPGCluster CR -> wave 5 (created after CRDs are established)
apiVersion: pgv2.percona.com/v2
kind: PerconaPGCluster
metadata:
annotations:
argocd.argoproj.io/sync-wave: "5"
name: argo-test
```

1.4 Create, sync, and automate

```
argocd app create percona-pg \
--repo https://github.com/<you>/<repo>.git \
--path gitops-argocd-postgresql \
--dest-server https://kubernetes.default.svc \
--dest-namespace default

argocd app sync percona-pg # manual roll-out
kubectl get pg # expect STATUS=ready, POSTGRES/PGBOUNCER counts

argocd app set percona-pg --sync-policy automated # continuous reconcile
```

Sync interval

ArgoCD polls Git every 180s by default; a committed change shows as **OutOfSync** until the next sync (or immediately under an automated policy). Tune in the `argocd - cm ConfigMap` if you need faster convergence.

2. Migration playbook A — backup & restore

Use case: move an existing PostgreSQL cluster onto the Percona operator by restoring from a pgBackRest backup. The new Percona cluster bootstraps from the existing archive before its first start. Fastest path when you can tolerate a short write-freeze; data written after the final backup and before cutover is lost. Tested with a Crunchy v5.8.x source and Percona v3.0.0 target, PostgreSQL 18 on both sides, over an S3-compatible store.

pgBackRest always uses HTTPS to S3

Even with `repo1-s3-verify-tls: "n"`, pgBackRest connects over HTTPS — that flag only skips certificate verification, it does not fall back to HTTP. For a self-signed in-cluster store you must still terminate TLS. This is the #1 cause of restore-job TLS failures.

2.1 Set namespace and shared repo credentials

```
export MIGRATION_NS=postgres-migration
kubectl create namespace $MIGRATION_NS

# secret carrying the S3/pgBackRest credentials both clusters use
kubectl apply -n $MIGRATION_NS -f 01-pgbackrest-secrets.yaml
```

2.2 Source repo settings (path-style S3)

The source cluster's pgBackRest `repo1-path` is the anchor of the whole migration — the target references the exact same path. For MinIO / SeaweedFS / Ceph you need path-style URIs:

```
global:
repo1-path: /crunchy-to-percona/repo1 # referenced verbatim by the target dataSource
repo1-s3-uri-style: path # required for path-style endpoints (MinIO, SeaweedFS)
repo1-s3-verify-tls: "n" # self-signed only; drop for AWS S3
repos:
- name: repo1
s3:
bucket: pg-migration
endpoint: <s3-endpoint>:8443
region: us-east-1
```

2.3 Take the cutover backup, then deploy the target operator

```
# freeze writes on the app side first for a consistent snapshot, then:
kubectl annotate postgrescluster <source> -n $MIGRATION_NS \
postgres-operator.crunchydata.com/pgbackrest-backup="$(date +%s)"

kubectl wait job --for=condition=Complete --timeout=600s \
--selector postgres-operator.crunchydata.com/pgbackrest-backup=manual,postgres-operator.crunchydata.com/cluster=<source> -n $MIGRATION_NS

kubectl apply -n $MIGRATION_NS --server-side \
-f https://raw.githubusercontent.com/percona/percona-postgresql-operator/refs/tags/v3.0.0/deploy/bundle.yaml
```

2.4 Bootstrap the Percona cluster from the backup

The restore is driven entirely by the `dataSource.pgbackrest` block. Point it at the source repo path; give the cluster's own backup repository a **different** path so future backups do not collide with the source archive.

```
spec:
  dataSource:
    pgbackrest:
      stanza: db
      configuration:
        - secret: { name: percona-pgbackrest-secret }
      global:
        repo1-path: /crunchy-to-percona/repo1 # MUST match the source exactly
        repo1-s3-uri-style: path
        repo1-s3-verify-tls: "n"
      repo:
        name: repo1
        s3: { bucket: pg-migration, endpoint: <s3-endpoint>:8443, region: us-east-1 }
      backups:
        pgbackrest:
          global:
            repo1-path: /percona-restored/repo1 # different path for the new cluster's own backups
```

2.5 Verify, baseline, cut over

```
kubectl wait perconapgcluster/<target> -n $MIGRATION_NS \
--for=jsonpath='{.status.state}'=ready --timeout=600s

# primary should NOT be in recovery -> expect 'f'
PRIMARY=$(kubectl get pod -n $MIGRATION_NS -o jsonpath='{.items[0].metadata.name}' \
--selector postgres-operator.crunchydata.com/cluster=<target>,postgres-operator.crunchydata.co
m/role=primary)
kubectl -n $MIGRATION_NS exec "$PRIMARY" -c database -- psql -t -c "SELECT
pg_is_in_recovery();"

# take a clean post-migration backup = new PITR baseline, independent of the source archive
kubectl apply -n $MIGRATION_NS -f 04-post-migration-backup.yaml

# repoint the app at the new PgBouncer service
kubectl get service -n $MIGRATION_NS \
--selector postgres-operator.crunchydata.com/cluster=<target>,postgres-operator.crunchydata.co
m/role=pgbouncer
```

Rollback is easy here

Until you delete the source cluster, rollback is just repointing the app connection string back to the source PgBouncer — the source primary still holds authoritative state because no writes hit the new cluster during cutover. Any writes sent to the new cluster after cutover must be replayed manually.

2.6 Troubleshooting — error signatures

archive.info missing

The target `dataSource.pgbackrest.global.repo1-path` does not match the source `repo1-path`. Diff them directly:

```
kubectl get postgrescluster <source> -n $MIGRATION_NS \
-o jsonpath='{.spec.backups.pgbackrest.global.repo1-path}'
kubectl get perconapgcluster <target> -n $MIGRATION_NS \
-o jsonpath='{.spec.dataSource.pgbackrest.global.repo1-path}'
```

Restore job fails with TLS errors

HTTPS reachability to the object store is broken (see the HTTPS callout above). Prove connectivity from inside the cluster:

```
kubectl run -i --rm s3-check --image=perconalab/awcli --restart=Never -n $MIGRATION_NS -- \
bash -c 'AWS_ACCESS_KEY_ID=<id> AWS_SECRET_ACCESS_KEY=<key> AWS_DEFAULT_REGION=us-east-1 \
aws --endpoint-url https://<s3-endpoint>:8443 --no-verify-ssl s3 ls s3://pg-migration'
```

Cluster stuck in 'restoring'

Read the pgBackRest restore-job logs directly:

```
kubectl logs -n $MIGRATION_NS -c pgbackrest \
--selector postgres-operator.crunchydata.com/cluster=<target>,postgres-operator.crunchydata.co
m/pgbackrest-restore=<target>
```

Data missing after restore

The restore only reaches the latest backup. If post-backup writes matter, re-freeze the source, take a fresh backup, then delete and recreate the target so it restores from the newer backup.

3. Migration playbook B — persistent volume reuse

Use case: keep the existing PGDATA volume and skip the data copy entirely. You retain the primary's PV, delete the source cluster, then create a Percona cluster whose PVC binds to that same PV; PostgreSQL starts on the existing data directory with no restore step. Best for very large volumes. This is the least rollback-friendly of the methods — see the rollback note.

3.1 Freeze writes and capture the PV identity

```
PRIMARY=$(kubectl get pod -n $MIGRATION_NS -o jsonpath='{.items[0].metadata.name}' \
--selector postgres-operator.crunchydata.com/cluster=<source>,postgres-operator.crunchydata.co
m/role=master)
PVC_NAME=$(kubectl get pod -n $MIGRATION_NS "$PRIMARY" \
-o jsonpath='{.spec.volumes[?(@.name=="postgres-data")].persistentVolumeClaim.claimName}')
PV_NAME=$(kubectl get pvc -n $MIGRATION_NS "$PVC_NAME" -o jsonpath='{.spec.volumeName}')
echo "pod=$PRIMARY pvc=$PVC_NAME pv=$PV_NAME"
```

3.2 Protect the PV, then delete the source

Dynamically provisioned PVs default to reclaim policy Delete — deleting the cluster would destroy the data. Flip it to Retain first.

```
kubectl patch pv "$PV_NAME" -p '{"spec":{"persistentVolumeReclaimPolicy":"Retain"}}'

# remove finalizer if the CR hangs on delete, then delete the source cluster + operator
kubectl patch postgrescluster <source> -n $MIGRATION_NS \
--type=json -p='[{"op":"remove","path":"/metadata/finalizers"}]' 2>/dev/null || true
kubectl delete postgrescluster <source> -n $MIGRATION_NS
```

A Retained PV goes to 'Released', not 'Available'

After the PVC is gone the PV keeps its old `claimRef` and cannot be re-bound. Clear the `claimRef` and wait for Available, then label it so the new PVC selector binds to it exclusively:

```
kubectl patch pv "$PV_NAME" --type=json -p='[{"op":"remove","path":"/spec/claimRef"}]'
kubectl wait pv "$PV_NAME" --for=jsonpath='{.status.phase}'=Available --timeout=60s
kubectl label pv "$PV_NAME" percona-pv-migration=migrated
```

3.3 Bind the new cluster to the retained PV

Start with a **single** replica and a label selector so the operator's PVC binds to your labelled PV rather than provisioning a fresh one. `pgBackRest` can use a local PVC-backed repo, so no S3 is required for this method.

```
spec:
instances:
- name: instance1
replicas: 1
dataVolumeClaimSpec:
selector:
matchLabels:
percona-pv-migration: migrated

kubectl wait perconapgcluster/<target> -n $MIGRATION_NS \
--for=jsonpath='{.status.state}'=ready --timeout=600s
kubectl -n $MIGRATION_NS exec "$PRIMARY" -c database -- psql -t -c "SELECT
pg_is_in_recovery();" # -> f
```

3.4 Scale out — and drop the selector first

Remove the PVC selector before scaling replicas

If the selector stays in place, new replica PVCs fail to provision because no other PV carries the migration label. Remove the selector and bump replicas in one patch:

```
kubectl patch perconapgcluster <target> -n $MIGRATION_NS --type=json -p='[
{"op":"remove","path":"/spec/instances/0/dataVolumeClaimSpec/selector"},
{"op":"replace","path":"/spec/instances/0/replicas","value":3}
]'
```

3.5 Troubleshooting — error signatures

PVC stuck in 'Pending'

The selector did not match the labelled PV. Check the label and the PV phase:

```
kubectl get pv "$PV_NAME" --show-labels
kubectl get pv "$PV_NAME" -o jsonpath='{.status.phase}'
```

PostgreSQL fails to start (data-directory errors)

Read the database container logs. If the source was shut down uncleanly there may be incomplete WAL; Patroni attempts crash recovery automatically — watch for progress.

```
kubectl -n $MIGRATION_NS logs "$PRIMARY" -c database
```

PV deleted before Retain was set

If the PV was removed with the default Delete policy, the data is gone and PV reuse is impossible — fall back to the backup-and-restore method (Part 2) from your most recent pgBackRest backup.

Rollback for PV reuse

Once the new cluster writes to PGDATA the original timeline is gone. If you need a way back, take a source-side pgBackRest backup *before* flipping the reclaim policy and treat that as your rollback point (recover into a fresh source cluster).

4. Major version upgrade (operator-managed)

The operator performs in-place major upgrades via a dedicated `PerconaPGUpgrade` custom resource — no manual `pg_upgrade` or dump/restore. Under the hood, when the CR is created the operator:

- adds a `pgv2.percona.com/allow-upgrade` annotation to the cluster CR to arm the upgrade;
- **pauses the cluster** — it is unavailable for the duration, so schedule maintenance;
- runs pods from a special image holding multiple PostgreSQL versions and executes `pg_upgrade` against the mounted data PVCs (copy, not dump/restore);
- takes a fresh full backup automatically once the upgrade completes.

Old backups only restore to the old version

Backups taken before the upgrade can restore *only* to the previous major version. The post-upgrade automatic full backup is your first recoverable baseline on the new version — confirm it succeeded before resuming traffic. Always test the upgrade in non-prod and take a full backup first.

The upgrade is triggered by applying a manifest naming the current and desired versions plus the target images. Consult the operator's upgrade guide for the exact `fromPostgresVersion` / `toPostgresVersion` and `image` fields for your release, as they have evolved across 2.4 → 3.x.

5. Forum incident library (annotated)

Real failures from the Percona Community Forum, each reduced to signature → cause → fix. These recur across operator versions because the operator wraps pgBackRest and Patroni.

5.1 Single-primary cluster: stanza never created

Symptom: a primary-only cluster (no replicas) never creates its pgBackRest stanza; on-demand backups fail. Operator log shows a pgBackRest exit code 31:

Error signature

```
command terminated with exit code 31: ERROR: [031]: configuration file contains duplicate options ('repo1-path', 'repo-path') in section '[global]'
```

Cause: a stray `/etc/pgbackrest.conf` baked into a custom image declared a conflicting `repo-path` alongside the operator-generated `repo1-path`. A second, related signature appears when the pgBackRest binary version differs between local and remote:

Related signature

```
command terminated with exit code 39: ERROR: [039]: expected value '2.44' for greeting key 'version' but got '2.43'
HINT: is the same version of pgBackRest installed on the local and remote host?
```

Fix

In the custom image, pin the pgBackRest package to the exact operator-expected version and remove the stray default config:

```
# Dockerfile
RUN rm -rf /etc/pgbackrest.conf
# and pin the package, e.g.
# percona-pgbackrest-1:2.43-1.el8.x86_64 (match the operator's bundled version)
```

Operational rule for custom images

When you build custom PostgreSQL images for the operator, keep the pgBackRest version **byte-for-byte aligned** with what the operator ships, and never let a base image inject its own `pgbackrest.conf` — the operator owns that configuration.

5.2 Restore does not revert data / archive-command fails

Symptom: a `--type=default` restore appears to run — recovery reaches a consistent state — but the database does not roll back to the intended point, then the pod shuts down. The tell-tale lines:

Log signature

```
WARN: unknown group 'postgres' in backup manifest mapped to current group
...
LOG: archive command failed with exit code 1
DETAIL: The failed archive command was: false
WARNING: archiving write-ahead log file "00000002.history" failed too many times
```

Reading it: the *unknown group 'postgres'* WARN is benign (a uid/gid mapping note, not the failure). The `archive-command-false` lines are expected during a restore bootstrap where archiving is intentionally disabled. The actual issue was the restore **type**: `--type=default` did not produce the intended reset,

whereas an explicit target did.

Fix

Use an explicit restore type. A point-in-time restore with `--type=time` (and a target timestamp) works reliably; for a full-backup restore use an explicit `full/immediate` type rather than relying on `default`.

```
# reliable forms
pgbackrest restore --type=time --target="2026-05-16 12:00:00+02" --stanza=db
pgbackrest restore --type=immediate --stanza=db # restore to end of the chosen backup
```

5.3 Quick-reference — other recurring signatures

Signature / symptom	Likely cause	First move
[032]: '' must be >= 1 character for 'set' option	restore CR missing/empty backup set selector	Populate the target backup name/set in the restore CR
Backup pod: container create error (2.8.x, OpenShift)	image/SCC or volume mount mismatch on the backup pod	<code>kubectl describe pod</code> the backup pod; check events + SCC
S3 backups worked on prior operator, break after upgrade	chart/operator bump changed pgBackRest repo fields	Diff old vs new CR; re-align repo1 S3 keys and paths
Operator/target pods never become Ready in custom namespace	RBAC: <code>pgo-target-role</code> not bound in that namespace	Verify Role/RoleBinding exist in the cluster namespace
<code>pgbackrest-config</code> ConfigMap not generated	stanza/repo misconfiguration blocks reconcile	Check operator logs for the reconcile error upstream of it

Appendix — diagnostic quick reference

A. First-response commands

```
kubectl get pg -n <ns> # cluster CR + status/counts
kubectl get perconapgcluster,perconapgbbackup,perconapgrestore -n <ns>
kubectl describe perconapgcluster <name> -n <ns> # events + condition messages
kubectl get pods -n <ns> -o wide # per-pod state / restarts
kubectl describe pod <pod> -n <ns> # init/containers, liveness, events
kubectl logs <pod> -n <ns> -c database # PostgreSQL / Patroni
kubectl logs <pod> -n <ns> -c pgbackrest # backup/restore engine
kubectl logs deploy/<operator> -n <ns> # operator reconcile loop
```

A CR that never reaches **Ready** almost always has its root cause one layer down: check the individual pods, then the database and pgbackrest containers, then the operator log for the reconcile error that precedes the visible symptom.

B. pgBackRest exit codes seen above

Code	Meaning in context	Typical trigger
[031]	Config error — duplicate/invalid option	stray pgbackrest.conf; conflicting repo path keys
[032]	Option value invalid	empty restore set selector in the restore CR
[039]	Version/handshake mismatch	local vs remote pgBackRest binary versions differ

C. Health assertions worth scripting

```
# primary accepts writes (not in recovery) -> expect f
psql -t -c "SELECT pg_is_in_recovery();"
# replica is streaming -> expect t on replicas, and a row per replica on the primary
psql -t -c "SELECT client_addr, state, sync_state FROM pg_stat_replication;"
# replication lag in bytes on a replica
psql -t -c "SELECT pg_wal_lsn_diff(pg_last_wal_receive_lsn(), pg_last_wal_replay_lsn());"
```

Signatures and commands reconstructed from Percona engineering blog posts and Percona Community Forum threads and rewritten as operational notes. Field names and image references are version-sensitive — validate against the docs for your operator release before running in production.